



Tarea 4: Safe RL (2026-S1)

Problema 1: Avance del Proyecto (1 Punto)

Para esta entrega se espera que presenten un avance equivalente de al menos el 30 % del proyecto final. Esto implica incluir una descripción clara y detallada de la metodología que utilizarán, así como resultados preliminares que evidencien el progreso.

Ejemplos de un buen avance (además de la descripción de la metodología):

- Análisis del simulador y resultados preliminares con un baseline.
- Pruebas iniciales de su controlador de RL sin técnicas adicionales (e.g. Solo PPO, solo TD3, sin domain randomization, sin model-based, sin técnicas de seguridad, etc)
- Estudio inicial de la función de recompensa, mostrando cómo distintas formulaciones afectan el comportamiento de un baseline.

Problema 2: Conducción Autónoma con Obstáculos (2 puntos)

Luego de estudiar varios métodos de RL, usted enfrentará una situación común en aplicaciones reales de control: el agente no solo debe lograr su objetivo, sino hacerlo sin violar restricciones de seguridad. En este problema, usted deberá entrenar un vehículo autónomo capaz de llegar a una meta evitando obstáculos utilizando métodos Lagrangianos.

El vehículo corresponde a un robot con tracción diferencial y una rueda loca delantera. Para esto se utilizará la librería *safety-Gymnasium*, la cual extiende la formulación clásica de RL al incorporar explícitamente los costos asociados a violaciones de seguridad.

A diferencia de un ambiente tradicional, en este caso el simulador no solo entrega una recompensa, sino también un costo de restricción. Este costo es distinto de cero cuando el agente choca con algún obstáculo. La interacción con el ambiente tiene la siguiente forma:

```
1 obs, reward, cost, terminated, truncated, info = env.step(action)
```

Código 1: Interacción con un ambiente de Safety-Gymnasium

Para esto, se le ha entregado un código de ayuda en `ppo.py`, el cual muestra cómo entrenar un agente PPO común y corriente sobre el ambiente. Este código puede ser modificado libremente, especialmente para mejorar gráficos, ajustar hiperparámetros, o registrar métricas adicionales.

La restricción del problema está dada por:

$$J_C(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T c_t \right] \leq d, \quad (1)$$

donde T corresponde al largo del episodio, c_t es el costo recibido en el instante t y d es el límite máximo permitido para el costo acumulado esperado. Para este problema considere:

$$d = 50 \quad (2)$$

Por otro lado, la función de recompensa está dada por:

$$r_t = (D_{t-1} - D_t) \quad (3)$$

Donde D es la distancia a la meta.

Hint: En `readme.txt` se muestra una forma en que se puede instalar *safety-Gymnasium*. La librería funciona con `python 3.8`, por lo que se recomienda crear un ambiente virtual con `conda` u otro método.

Su Tarea

- a) (3 décimas) Como primer paso, ejecute el código base entregado en `ppo.py` sin modificar el algoritmo principal. Grafique el retorno acumulado y el costo acumulado por episodio (junto con el límite d). Comente sus resultados, en particular si existe un conflicto entre maximizar recompensa y reducir violaciones de seguridad.
- b) (10 décimas) Utilizando el código entregado como base, implemente PPO-Lagrangian (PPO-Lag). Luego:
- Explique brevemente el algoritmo.
 - Indique qué cambios realizó sobre PPO original.
 - Grafique el retorno y el costo acumulado por episodio (junto con el límite d).
 - Grafique la evolución del multiplicador λ durante el entrenamiento.
- Discuta si el método logra reducir el costo acumulado bajo el umbral $d = 50$ y qué ocurre con el retorno al imponer la restricción.
- c) (5 décimas) Ahora, implemente PID-Lagrangian (PID-Lag). Luego:
- Explique las diferencias entre PPO-Lagrangian y PID-Lagrangian
 - Grafique el retorno y el costo acumulado por episodio.
 - Grafique la evolución del multiplicador λ durante el entrenamiento.
- d) (2 décimas): Finalmente, compare los métodos implementados y discuta los resultados obtenidos.

Problema 3: Control de Glucosa con CBFs (3 Puntos)

Tras una vida de excesos, Alan Brito comenzó una mañana como cualquier otra: tres panes, una torta y un café con cuatro cucharadas de azúcar. Sin embargo, algo no estaba bien. A mitad del día empezó a sentirse inusualmente cansado y débil, tanto que decidió (por primera vez en mucho tiempo) ir al médico. Después de varios exámenes, el doctor le dio el diagnóstico: Contra todo pronóstico, y pese a lo que cualquiera habría pensado basándose en su dieta, Alan Brito no tenía diabetes Tipo 2, sino diabetes Tipo 1 ¹

Una vez más, Alan Brito recurre a usted por sus conocimientos en aprendizaje reforzado para que lo ayude a manejar su glucosa sanguínea mediante un páncreas artificial. Para ello pone a su disposición un simulador del sistema glucorregulatorio de personas con diabetes Tipo 1 ², con el objetivo de resolver el problema de control de la glucosa basal.

En la mayoría de las soluciones de páncreas artificial, la administración de insulina se aborda mediante una combinación de control *feedforward* y control de *feedback*. El componente *feedforward* actúa cuando la persona anuncia una comida, administrando un bolo de insulina proporcional a la cantidad estimada de carbohidratos ingeridos. Este bolo se calcula utilizando parámetros individuales del paciente, estimados a partir de exámenes médicos y ajustes clínicos.

Por otro lado, el controlador de *feedback* modifica la insulina basal, es decir, la dosis de insulina administrada de manera continua durante el día. Su objetivo es corregir desviaciones de la glucosa respecto al rango deseado, compensando perturbaciones no modeladas o no anunciadas, como estrés, ejercicio, variaciones fisiológicas, o compensaciones imperfectas del controlador *feedforward*.

Generalmente, se considera que la glucosa se encuentra en un rango seguro entre 70 y 180 mg/dL. Sin embargo, violar estos límites tiene efectos asimétricos. Por un lado, niveles de glucosa superiores a 180 mg/dL pueden producir daño a largo plazo en ciertos órganos, pero no suelen representar un riesgo inmediato de muerte. Por otro lado, niveles inferiores a 70 mg/dL pueden ser peligrosos e incluso mortales. Por esta razón, cualquier solución de páncreas artificial debe incorporar mecanismos de seguridad que eviten poner en riesgo la vida de las personas.

¹<https://www.mayoclinic.org/es/diseases-conditions/type-1-diabetes/symptoms-causes/syc-20353011>

²<https://github.com/jxx123/simglucose>

En esta tarea, usted deberá diseñar un controlador de insulina basal utilizando RL, asumiendo que un controlador experto se encarga de administrar los bolos de insulina al momento de las comidas. Además, deberá diseñar una *Control Barrier Function* (CBF) para imponer una restricción dura sobre el límite inferior de la glucosa.

Para esto, se le han entregado varios scripts de ayuda:

- `expert_controller.py`: Contiene el código del controlador experto. Este archivo no debe ser modificado.
- `hovorka.py`: Contiene el código de un modelo imperfecto y simplificado de la planta, que puede ser utilizado para diseñar el controlador y definir la función de barrera.
- `KF.py`: Contiene un filtro de Kalman lineal y discreto, que permite estimar los estados del modelo de Hovorka a partir de mediciones ruidosas de la planta.
- `main.py`: Contiene un ejemplo de cómo instanciar la planta en *simglucose* para distintas personas, obtener las matrices linealizadas del modelo de Hovorka, realizar las transformaciones de unidades correspondientes (Hovorka utiliza unidades distintas) y estimar los estados de Hovorka utilizando el filtro de Kalman.

Su Tarea

- a) (10 décimas) Entrene un agente de RL para controlar la insulina basal en los 10 adultos del simulador, sin incorporar mecanismos adicionales de seguridad. Puede basarse en los códigos de PPO, SAC o TD3 entregados anteriormente. Para esto deberá:

- Diseñar una función de recompensa. Puede utilizar la recompensa del simulador o el índice de riesgo disponible en `info['risk']`. En caso de utilizar alguna de estas opciones, debe explicarla y justificar su elección.
- Definir el estado Markoviano utilizado por el controlador. Considere que este sistema presenta retardos importantes.
- Graficar el retorno acumulado durante el entrenamiento y la evolución de las métricas TIR (*Time in Range*, 70–180 mg/dL), TBR (*Time Below Range*), TAR (*Time Above Range*) y Glucosa Promedio. Reporte estas métricas tanto en promedio como separadas por adulto.

Considere que cada día corresponde a un episodio distinto. En `main.py` se muestra un ejemplo de cómo aleatorizar las comidas.

- b) (10 décimas) Diseñe una función de barrera discreta para imponer la restricción de seguridad sobre el límite inferior de la glucosa. Para esto deberá:

- Explicar paso a paso el desarrollo matemático de su solución.
- Escribir de manera explícita el problema de optimización a resolver, incluyendo la restricción impuesta por la función de barrera.
- Implementar la *safety layer* correspondiente y aplicarla sobre la política ya entrenada en el punto anterior.
- Graficar, para algunos días de prueba y para al menos un adulto seleccionado, la evolución de la glucosa, la acción propuesta por el controlador, la acción modificada por la *safety layer* y los valores de la función de barrera.
- Evaluar el desempeño durante una semana de prueba y calcular las métricas TIR, TBR y TAR para cada adulto.

- c) (10 décimas) Finalmente, entrene una nueva política incorporando la función de barrera durante el proceso de entrenamiento (puede aplicar otras de las mejoras vistas en clase).

Grafique las mismas métricas solicitadas en el punto a): retorno acumulado, TIR, TBR y TAR, tanto en promedio como diferenciando por adulto.

Compare los resultados obtenidos en este punto con los del punto b), donde la *safety layer* se utilizó únicamente durante la evaluación. Discuta los efectos de incorporar la barrera en el entrenamiento.

Hint: Recuerde que no es necesario resolver el problema de optimización en cada paso. Primero verifique si la condición de barrera se cumple con la acción original. Si la condición se satisface, la acción puede aplicarse directamente y no es necesario modificarla mediante la *safety layer*.

Anexo

Nota de la tarea

La nota de esta tarea estará dada por las condiciones del siguiente **sistema experto**:

- IF $N_{T_4} \geq 4$ THEN $NF_{T_4} = \max(N_{T_4}, NP)$
- IF $N_{T_4} < 4$ THEN $NF_{T_4} = N_{T_4}$

Donde:

- NT_4 : Nota preliminar de esta tarea
- NP : Nota promedio de las tareas anteriores, es decir $NP = \frac{N_{T_1} + N_{T_2} + N_{T_3}}{3}$
- NF_{T_4} : Nota final de esta tarea.

Informe

El informe debe ser realizado en Latex, utilizando el template que está en Canvas.

Código

- Todo su código debe ser realizado en python version ≥ 3.11 .
- Está estrictamente prohibido el uso de Chatbots para completar su tarea. Si el profesor o los ayudantes determinan que utilizó un chatbot durante el desarrollo la tarea se calificará con Nota 1 (Ver diapositivas de la primera clase para el detalle de los lineamientos del curso en este tema).